

Guia de Actividad: HADA con GitHub y C#

Esta actividad os ayudara a reactivar el equipo con una entrega real en GitHub: desarrollar un programa pequeno en C# por ramas y fusionar el trabajo mediante Pull Requests.

” Objetivo

Construir en equipo una aplicacion de consola en C# para gestionar tareas, aplicando flujo de organizacion GitHub, ramas y revision final segun roles H.A.D.A.



Recursos

- Ordenadores con VS Code.
- Cuenta GitHub de cada miembro.
- SDK de .NET instalado (version 8 o superior).

Repositorio objetivo en la organizacion: `hada-task-tracker`

Roles del Equipo: H.A.D.A.

Hacedor(a) (H)

Implementa código, hace commits frecuentes y mantiene su rama lista para revisión.

Analista (A)

Define criterios de aceptación por tarea, estima tiempos y revisa si el resultado cumple lo acordado.

Diplomatico(a) (D)

Coordina el tablero de trabajo, ordena prioridades y gestiona bloqueos entre ramas o PR.

Arquitecto(a) (A)

Diseña la estructura del programa, define nombres de ramas y valida la integración final en `main`.

Producto a construir

Aplicacion de consola `TaskTracker` con estas funciones minimas:

- Anadir tarea.
- Listar tareas.
- Marcar tarea como completada.
- Guardar/cargar tareas en fichero `tareas.json`.

Plan en 2 sesiones

Sesion 1: Preparar, dividir y avanzar por ramas (50 min)

Bloque 1: Preparacion del repositorio (15 min)

1. En la organizacion, cread el repositorio `hada-task-tracker` con `README.md`.
2. Clonad el repositorio en local.
3. Inicializad el proyecto C#:

```
git clone https://github.com/ORGANIZACION/hada-task-tracker.git
cd hada-task-tracker
dotnet new console -n TaskTracker
git add .
git commit -m "Inicializa proyecto TaskTracker"
git push origin main
```

Bloque 2: Plan HADA y reparto por ramas (15 min)

1. Definid 4 tareas técnicas, por ejemplo:
 - * T1: modelo de tarea (`TaskItem`) y lista en memoria.
 - * T2: menu por consola y opcion "anadir tarea".
 - * T3: listar y completar tareas.
 - * T4: persistencia JSON (`tareas.json`).
2. Cread una rama por tarea:

```
git switch -c feature/modelo-taskitem
git switch main
git switch -c feature/menu-anadir
git switch main
git switch -c feature/listar-completar
git switch main
git switch -c feature/persistencia-json
git switch main
git push --all origin
```

3. Asignad una tarea por persona segun rol y estimad tiempo por tarea.

Bloque 3: Primer desarrollo en paralelo (20 min)

Cada miembro empieza su tarea en su rama y sube al menos un commit funcional.

Entregables de la Sesion 1

1. Repositorio creado en la organizacion y proyecto C# inicial en `main`.
2. Cuatro ramas de funcionalidad publicadas en remoto.
3. Minimo 1 commit por rama (o evidencia de avance en 3 ramas como minimo).
4. Fichero `PLAN_HADA.md` con reparto de tareas, estimaciones y responsable.

Sesion 2: Integrar, validar y evaluar (50 min)

Bloque 1: Finalizacion de tareas y PR (20 min)

1. Terminad el desarrollo pendiente en cada rama.
2. Cada rama abre un PR contra `main`.
3. Revisiones minimas por PR:
 - * Compila (`dotnet build`).
 - * Funcion principal probada en consola.
 - * Mensaje de commit claro.

Bloque 2: Fusion ordenada y prueba de integracion (20 min)

1. Fusionad PR en este orden recomendado: T1 -> T2 -> T3 -> T4.
2. Tras cada fusion, ejecutad en `main` :

Bloque 3: Evaluacion HADA (10 min)

Rellenad `RETRO_HADA.md` con estas evidencias:

1. Viabilidad tecnica (Analista):

- * Que funcione bien del diseno.
- * Que deuda tecnica queda.

2. Temporizacion real vs estimada (Arquitecto + Analista):

- * Tiempo estimado por tarea.
- * Tiempo real por tarea.
- * Desviacion y causa.

3. Colaboracion y flujo GitHub (Diplomatico):

- * Numero de PR abiertos/cerrados.
- * Bloqueos detectados y como se resolvieron.

4. Ejecucion (Hacedor):

- * Que entregables quedaron finalizados en `main`.
- * Que tarea se aplaza al siguiente sprint.

Entregables de la Sesion 2

1. PR de cada funcionalidad revisado y fusionado.
2. Aplicacion `TaskTracker` funcionando en `main` con las 4 funciones minimas.
3. Fichero `RETRO_HADA.md` completo con viabilidad, tiempos y mejora.
4. Demo corta del equipo ejecutando la aplicacion en consola.



Resultado esperado

El equipo termina con una aplicacion C# funcional, historial de ramas y PR, y una revision objetiva de viabilidad, tiempos y forma de trabajo segun H.A.D.A.

El objetivo final es que podais repetir este flujo en proyectos reales: planificar, desarrollar en paralelo, integrar con calidad y aprender del resultado.